

3168. Minimum Number of Chairs in a Waiting Room

[Easy](#) [Topics](#) [Companies](#) [Hint](#)

You are given a string `s`. Simulate events at each second `i`:

- If `s[i] == 'E'`, a person enters the waiting room and takes one of the chairs in it.
- If `s[i] == 'L'`, a person leaves the waiting room, freeing up a chair.

Return the **minimum** number of chairs needed so that a chair is available for every person who enters the waiting room given that it is initially **empty**.

Example 1:

Input: `s = "EEEEEEE"`

Output: 7

Explanation:
After each second, a person enters the waiting room and no person leaves it. Therefore, a minimum of 7 chairs is needed.

Example 2:

Input: `s = "ELELEEL"`

Output: 2

Explanation:
Let's consider that there are 2 chairs in the waiting room. The table below shows the state of the waiting room at each second.

`</>` Code

Java [Auto](#)

```
1 class Solution {
2     public int minimumChairs(String s) {
3         int currentPeople = 0;
4         int maxChairsNeeded = 0;
5         for (int i = 0; i < s.length(); i++) {
6             char event = s.charAt(i);
7             if (event == 'E') {
8                 currentPeople++;
9             } else if (event == 'L') {
10                currentPeople--;
11            }
12            if (currentPeople > maxChairsNeeded) {
13                maxChairsNeeded = currentPeople;
14            }
15        }
16
17        return maxChairsNeeded;
18    }
19 }
```

Saved Ln 4, Col 33

☒ Testcase | [Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2 ☒ Case 3

Input

s =
"EEEEEEE"

Output

1446. Consecutive Characters

Easy

Topics

Companies

Hint

The **power** of the string is the maximum length of a non-empty substring that contains only one unique character.

Given a string `s`, return *the power of* `s`.

Example 1:

Input: `s = "leetcode"`

Output: 2

Explanation: The substring "ee" is of length 2 with the character 'e' only.

Example 2:

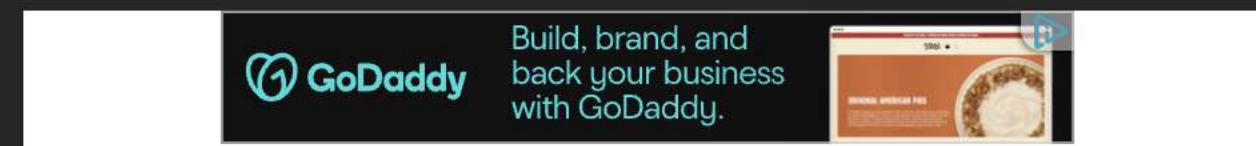
Input: `s = "abbcccddeeecba"`

Output: 5

Explanation: The substring "eeeee" is of length 5 with the character 'e' only.

Constraints:

- `1 <= s.length <= 500`
- `s` consists of only lowercase English letters.



Code

Java Auto

```
7      int currentStreak = 1;
8      for (int i = 1; i < s.length(); i++) {
9          if (s.charAt(i) == s.charAt(i - 1)) {
10             currentStreak++;
11         } else {
12             currentStreak = 1;
13         }
14         if (currentStreak > maxPower) {
15             maxPower = currentStreak;
16         }
17     }
18
19     return maxPower;
20 }
21 }
```

Saved

Ln 5, Col 10

☒ Testcase | >_ Test Result**Accepted** Runtime: 0 ms☒ Case 1☒ Case 2

Input

s =
"leetcode"

Output

2

1732. Find the Highest Altitude

Easy

Topics

Companies

Hint

There is a biker going on a road trip. The road trip consists of $n + 1$ points at various altitudes. The biker starts his trip on point 0 with altitude equal 0 .

You are given an integer array `gain` of length `n` where `gain[i]` is the **net gain in altitude** between points `i` and `i + 1` for all ($0 \leq i < n$). Return *the highest altitude of a point*.

Example 1:

Input: `gain = [-5,1,5,0,-7]`

Output: `1`

Explanation: The altitudes are `[0,-5,-4,1,1,-6]`. The highest is `1`.

Example 2:

Input: `gain = [-4,-3,-2,-1,4,3,2]`

Output: `0`

Explanation: The altitudes are `[0,-4,-7,-9,-10,-6,-3,-1]`. The highest is `0`.

Constraints:

- $n == \text{gain.length}$
- $1 \leq n \leq 100$
- $-100 \leq \text{gain}[i] \leq 100$

Code

Java | Auto

```
1 class Solution {
2     public int largestAltitude(int[] gain) {
3         int currentAltitude = 0;
4         int maxAltitude = 0;
5         for (int i = 0; i < gain.length; i++) {
6             currentAltitude += gain[i];
7             if (currentAltitude > maxAltitude) {
8                 maxAltitude = currentAltitude;
9             }
10        }
11
12        return maxAltitude;
13    }
14 }
```

Saved

Ln 4, Col 30

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1

Case 2

Input

gain =
[-5,1,5,0,-7]

Output

1

Expected

1480. Running Sum of 1d Array

Easy

[Topics](#)[Companies](#)[Hint](#)

Given an array `nums`. We define a running sum of an array as `runningSum[i] = sum(nums[0]...nums[i])`.

Return the running sum of `nums`.

Example 1:

Input: `nums = [1,2,3,4]`

Output: `[1,3,6,10]`

Explanation: Running sum is obtained as follows: `[1, 1+2, 1+2+3, 1+2+3+4]`.

Example 2:

Input: `nums = [1,1,1,1,1]`

Output: `[1,2,3,4,5]`

Explanation: Running sum is obtained as follows: `[1, 1+1, 1+1+1, 1+1+1+1, 1+1+1+1+1]`.

Example 3:

Input: `nums = [3,1,2,10,1]`

Output: `[3,4,6,16,17]`

Constraints:

- `1 <= nums.length <= 1000`
- `-106 <= nums[i] <= 106`

</> Code

Java [Auto](#)[≡](#) [🔖](#) [{ }](#) [↶](#) [↷](#)

```
1 class Solution {
2     public int[] runningSum(int[] nums) {
3         for (int i = 1; i < nums.length; i++) {
4             nums[i] = nums[i] + nums[i - 1];
5         }
6         return nums;
7     }
8 }
```

Saved

Ln 3, Col 48

[Testcase](#) | [Test Result](#)[⌂](#) [^](#)**Accepted** Runtime: 0 ms[Case 1](#)[Case 2](#)[Case 3](#)

Input

nums =
[1,2,3,4]

Output

[1,3,6,10]

Expected